Guardrails — Cost Caps, Depth Limits, Hallucination Checks

SPEC-TH-020 v0.1 DRAFT Jeremie Nunez

1 Context

The Thalamus orchestrator drives recursive LLM-backed cortices that query internal data, external sources and other cortices. Without a hard safety envelope, such loops can burn unbounded tokens, recurse without progress, or surface fabricated claims.

This spec defines **Guardrails**: the non-bypassable control layer that bounds every cortex invocation and every research chain. Enforcement lives in the orchestrator, not in the cortex prompts — prompts can drift, code cannot. This is a sovereignty-critical surface: it is where operators retain control of cost, depth, and factuality against non-deterministic LLM behaviour.

Guardrails sit between the cortex executor (`packages/thalamus/src/cortices/executor.ts`) and the cortex LLM transport. All cost, depth and validation decisions pass through it; none of them are delegated to the model.

2 Goals

- Enforce a per-cortex **token and monetary budget** at call time, independent of what the cortex prompt asks for.
- Enforce a **depth limit** on any recursive or chained invocation, measured in iterations and in cumulative cost.
- Detect **hallucinations** via structured-output validation, schema conformance and source-citation requirements; reject or quarantine findings that fail.
- On breach, always surface a **partial result with an explicit breach marker**; never silently truncate.
- Emit **observability signals** (metrics, structured logs) on every breach and every quarantine, so operators can audit non-deterministic behaviour post-hoc.

3 Non-Goals

Non-Goal

This spec does not define:

- The content of cortex prompts (see SPEC-TH-011, cortex skills).
- Prompt-injection sanitation of inbound data payloads (see SPEC-TH-021, input sanitiser). Guardrails here govern *orchestrator behaviour*, not *payload cleanup*.
- Rate limiting at the HTTP edge (see SPEC-API-030).
- Long-term cost accounting across tenants (see SPEC-BILL-010).
- Model selection, fallback strategy, or retry policy at the transport layer (see SPEC-TH-014).

4 Invariants

Invariant

INV-1 — Budget is non-bypassable. Every cortex invocation is wrapped by `withGuardrails(budget, fn)`. It is impossible for a cortex to reach the LLM transport without a resolved, typed `Budget` object. Calling the transport directly is a lint-enforced violation.

Invariant

INV-2 — Budget is monotonic. Cumulative token and monetary cost on a chain only ever increase; there is no reset, discount or retroactive rebate across iterations of the same research chain. Reported cost $\geq$ actual cost for any prefix of the chain.

Invariant

INV-3 — Depth is bounded by construction. A research chain terminates in at most `maxIterationsPerChain` cycles, regardless of the novelty signal, LLM confidence, or cortex opinion. The orchestrator decrements the remaining-depth counter before dispatching each iteration; depth 0 short-circuits to a partial result.

Invariant

INV-4 — Breach is observable. On every breach (budget, depth, validation), the result carries a `breach` field enumerating the triggered guard(s), the measured value and the threshold. A silent success path on breach is a defect.

Invariant

INV-5 — Unverifiable findings do not reach consumers. A finding without a valid schema shape, or without at least one non-empty `evidence[]` citation pointing to an allow-listed source, is quarantined with `findingType = rejected_unsourced` and never emitted to downstream stores.

5 Interface

The orchestrator exposes a single entry point that all cortex calls must flow through. The contract is total: every outcome maps to a typed result, including breaches.

```
// Budget resolved per complexity tier (simple | moderate | deep)
export interface Budget {
  maxIterations: number;    // hard cap on chain depth
  maxCost: number;          // monetary cap per chain, USD
  maxTokens: number;        // aggregate prompt+completion tokens
  maxWallClockMs: number;   // chain timeout
  confidenceTarget: number;
  coverageTarget: number;
}

export type BreachKind =
  | "budget_cost"
  | "budget_tokens"
  | "depth_iterations"
  | "wall_clock"
  | "schema_invalid"
```

```
  | "unsourced_finding"
  | "source_not_allowlisted";

export interface Breach {
  kind: BreachKind;
  measured: number | string;
  threshold: number | string;
  iteration: number;
  cortex: string;
}

export interface GuardedOutput<T> {
  result: T;                    // partial if breach.length > 0
  partial: boolean;
  breaches: Breach[];
  usage: { tokens: number; costUsd: number; iterations: number; };
}

export function withGuardrails<T>(
  budget: Budget,
  ctx: ChainContext,
  fn: (remaining: Budget) => Promise<T>,
): Promise<GuardedOutput<T>>;

export function validateFinding(
  raw: unknown,
  allowlist: SourceAllowlist,
): Finding | RejectedFinding;
```

Emitted metrics (Prometheus-compatible):

- `thalamus_guardrail_breach_total{kind,cortex}` — counter
- `thalamus_chain_cost_usd{cortex,tier}` — histogram
- `thalamus_chain_iterations{cortex,tier}` — histogram
- `thalamus_finding_rejected_total{reason,cortex}` — counter

6 Scenarios

Scenario: S-1 — Nominal within budget

Given a chain with `budget = simple` ($0.03, 2 iter)
When iteration 1 costs $0.008, produces 3 valid findings, and the novelty signal says stop
Then the chain exits normally with `partial=false`, `breaches=[]`, and `usage.costUsd=0.008`.

Scenario: S-2 — Monetary cap exceeded mid-chain

Given `budget.maxCost = 0.06` (moderate tier)
And iterations 1–3 accumulated $0.055
When iteration 4 is about to dispatch and the transport's pre-flight cost estimate is $0.012
Then the orchestrator refuses to dispatch iteration 4, returns `GuardedOutput` with `partial=true`, `breaches=[{kind:"budget_cost", measured:0.055, threshold:0.06}]`, and includes all findings accumulated through iteration 3.

And metric `thalamus_guardrail_breach_total{kind="budget_cost"}` is incremented once.

Scenario: S-3 — Token cap exceeded during iteration

Given a running iteration with `maxTokens=60 000` remaining
When the streaming LLM response crosses 60 000 cumulative tokens mid-stream
Then the transport stream is cancelled at the next token boundary
And a partial finding set is parsed from the already-received prefix; any incomplete JSON object is dropped (not reconstructed)
And the result carries `breaches=[{kind:"budget_tokens",...}]`.

Scenario: S-4 — Depth limit reached

Given `maxIterationsPerChain=8` and the chain at iteration 8 with an active novelty signal
When the strategist cortex requests a 9th iteration
Then the request is denied regardless of remaining cost budget
And the output is marked `partial=true` with `kind:"depth_iterations"`.

Scenario: S-5 — Hallucination: finding without evidence

Given a cortex returns a finding with `evidence=[]`
When `validateFinding` runs
Then the finding is rejected, replaced by a `RejectedFinding` stub carrying `reason="unsourced"`
And it is *not* persisted to the knowledge graph
And metric `thalamus_finding_rejected_total{reason="unsourced"}` is incremented.

Scenario: S-6 — Hallucination: non-allowlisted source

Given a finding cites a URL whose host is not in the per-cortex source allowlist
When validation runs
Then the finding is rejected with `reason="source_not_allowlisted"` and the offending URL is logged at `warn`.
And the remainder of the finding set for that iteration is kept.

Scenario: S-7 — Schema-invalid structured output

Given the LLM returns JSON that fails the cortex output Zod schema (missing `confidence` field)
When the validator runs after one retry with schema-repair prompt
Then on second failure, the iteration returns zero findings with `breaches=[{kind:"schema_invalid"}]`
And cost of the failed iteration is still charged to the chain budget (INV-2).

Scenario: S-8 — Wall-clock timeout on a long iteration

Given a chain-level `maxWallClockMs=300 000`
When the current iteration's transport call exceeds 90 000 ms (`cortex.timeoutMs`) or the aggregate exceeds 300 000 ms
Then the in-flight call is aborted via `AbortSignal`; partial findings from prior iterations are returned with `kind:"wall_clock"`.

Scenario: S-9 — Consecutive zero-finding iterations

Given `consecutiveZeroStop=2`
When iteration n and iteration $n+1$ both produce zero valid findings after validation
Then the chain terminates with `partial=false` (not a breach — a normal stop condition)
And a structured log entry tags the stop reason as `"diminishing_returns"`. This is *not* a breach.

7 Acceptance Criteria

Acceptance Criterion AC-1

Given a chain where cumulative cost crosses `budget.maxCost` after iteration k, the orchestrator MUST NOT dispatch iteration $k+1$. The returned `GuardedOutput.partial` MUST be `true` and `breaches` MUST contain exactly one entry with `kind="budget_cost"`.

Acceptance Criterion AC-2

Given a chain at iteration equal to `budget.maxIterations`, any further dispatch MUST be refused with a `depth_iterations` breach. This holds even if remaining cost budget is non-zero.

Acceptance Criterion AC-3

Given a finding with `evidence.length === 0`, `validateFinding` MUST return a `RejectedFinding` and the finding MUST NOT appear in the persisted knowledge graph.

Acceptance Criterion AC-4

Given a finding whose `evidence[i].url` host is not in the cortex's source allowlist, `validateFinding` MUST return a `RejectedFinding` with `reason="source_not_allowlisted"`.

Acceptance Criterion AC-5

Given an LLM response that fails schema validation twice in a row for the same iteration, that iteration MUST yield zero findings and MUST still increment `usage.costUsd` with the attempted cost (monotonicity, INV-2).

Acceptance Criterion AC-6

On every breach, exactly one increment of `thalamus_guardrail_breach_total` with the matching `kind` and `cortex` labels MUST be observed. No breach MUST be silent; a unit test asserting breach-without-metric MUST fail the build.

Acceptance Criterion AC-7

`GuardedOutput` MUST always be returned, even on breach. The orchestrator MUST NOT `throw` out of `withGuardrails` for any of the defined `BreachKinds`; thrown exceptions are reserved for infrastructure errors (DB unreachable, transport crash).

Acceptance Criterion AC-8

Given a wall-clock overrun, the in-flight transport call MUST receive an `AbortSignal.abort()` within 500 ms of the deadline, verified by an integration test that measures dispatch-to-abort latency.

Acceptance Criterion AC-9

Partial results returned on breach MUST be structurally identical (same TypeScript type, same JSON shape) to successful results; consumers MUST be able to rely on `result` being safe to read regardless of `partial`'s value.

Acceptance Criterion AC-10

The per-tier `Budget` (`simple`, `moderate`, `deep`) MUST be the *only* source of numeric caps at runtime. No literal cost, iteration or token number MUST appear inside cortex prompts or cortex skill code — enforced by a repository-level grep test.

8 Traceability

AC	Test file	Test name
AC-1	`packages/thalamus/test/guardrails.cost.spec.ts`	`aborts chain on cost cap`
AC-2	`packages/thalamus/test/guardrails.depth.spec.ts`	`refuses dispatch at max depth`
AC-3	`packages/thalamus/test/guardrails.validate.spec.ts`	`rejects unsourced finding`
AC-4	`packages/thalamus/test/guardrails.validate.spec.ts`	`rejects non-allowlisted source`
AC-5	`packages/thalamus/test/guardrails.validate.spec.ts`	`charges cost on schema-invalid`
AC-6	`packages/thalamus/test/guardrails.metrics.spec.ts`	`emits one breach metric per breac`
AC-7	`packages/thalamus/test/guardrails.contract.spec.ts`	`never throws on defined breach ki`
AC-8	`packages/thalamus/test/guardrails.timeout.spec.ts`	`aborts in-flight call within 500m`
AC-9	`packages/thalamus/test/guardrails.contract.spec.ts`	`partial result has same shape`
AC-10	`packages/thalamus/test/guardrails.nomagic.spec.ts`	`no literal caps outside config`

9 Open Questions

- Should the cost cap be checked *before* dispatch using a pre-flight token estimate, or only *after* dispatch on reported usage? Current decision: pre-flight estimate + post-flight reconciliation. Needs validation against transport accuracy.
- Per-tenant vs. per-chain monetary caps: in scope for SPEC-BILL-010, out of scope here — but the interface must not preclude tenant-level wrapping.
- Allowlist governance: who owns the per-cortex source allowlist? Proposed: declared in the cortex skill header, reviewed at skill-registration time. To be confirmed.